GUIA DE APRESENTAÇÃO ORAL

ROTEIRO PEDIDOJÁ

Falas prontas, demonstração do sistema e respostas a perguntas

Duração sugerida: 8–10 minutos • Grupos com 2 ou 3 integrantes

Integrantes: _____

Professor(a): _____

Instituição / turma: _____

2026

Tempo recomendado: 8 a 10 minutos + perguntas. **Integrantes:** preencher nomes. **Público:** docente e turma. **Objetivo:** demonstrar um fluxo completo, com a API, as integrações externas, o banco e a segurança. Não afirmar que a implantação AWS está concluída sem evidência real.

Slide 1 — Abertura (0:00–0:45)

Quem fala: integrante 1.

“Boa noite. Nosso projeto se chama **PedidoJá** e corresponde ao tema três do Projeto Integrador: status de pedido em um e-commerce. A ideia surgiu de um problema bem comum: o cliente faz uma compra e precisa entrar no site ou perguntar à loja várias vezes para saber se o pagamento foi confirmado, se o pedido foi separado ou se já saiu para entrega. Nosso sistema automatiza esse acompanhamento, com notificações pelo Telegram.”

Slide 2 — Problema, proposta e usuários (0:45–1:40)

Quem fala: integrante 1.

“Temos dois atores principais: o administrador da loja, que cadastra produtos, clientes e pedidos; e o consumidor, que recebe as atualizações. Ao invés de mensagens manuais, cada mudança de etapa gera um evento no banco e uma tentativa de envio automático. As etapas são, obrigatoriamente, **recebido, pago, separado, enviado e entregue**. O histórico permite comprovar quando cada etapa aconteceu.”

Slide 3 — Tecnologias e arquitetura (1:40–2:45)

Quem fala: integrante 2.

“Para a API REST utilizamos Node.js com Express. O banco é PostgreSQL e sua estrutura é criada por migrations versionadas. Para validação do endereço usamos o ViaCEP, uma API pública que retorna os dados do CEP. A comunicação com o cliente ocorre por Telegram Bot API. Nossa aplicação tem dois processos: a API responde às requisições HTTP e um worker independente processa mensagens de entrada e saída. Tudo é organizado por Docker Compose no ambiente local. Para produção, escolhemos a AWS com EC2 e RDS PostgreSQL, Route 53 para DNS e Caddy com HTTPS.”

Slide 4 — Demonstração 1: cliente e produto (2:45–3:50)

Quem fala: integrante 2, com tela compartilhada.

“Vamos mostrar o uso do sistema. Aqui cadastramos um produto com preço em reais e um cliente com CEP. Ao enviar o formulário, nossa API consulta o ViaCEP e salva o endereço normalizado. O cadastro também cria um convite exclusivo para o cliente ativar o Telegram, porque bots não podem mandar mensagens para pessoas que nunca iniciaram conversa com eles.”

Ações na tela: *abrir o painel, cadastrar produto, cadastrar cliente, exibir endereço preenchido e link gerado. Caso tenha bot ativo, abrir o link no celular e tocar em INICIAR.* ***Não mostrar `ADMIN_API_KEY`, tokens ou senhas.***

Slide 5 — Demonstração 2: novo pedido (3:50–5:10)

Quem fala: integrante 2.

“Agora selecionamos o cliente, um produto e a quantidade. O valor total não é calculado a partir de um preço enviado pelo navegador: o backend consulta o valor no banco. O pedido é criado no estado *recebido* e já gera um registro de notificação. Isso garante que cada pedido tenha histórico e que as mensagens possam ser auditadas.”

Ações na tela: *criar pedido; mostrar número, status e total; se Telegram vinculado, exibir a mensagem recebida.*

Slide 6 — Demonstração 3: cinco etapas e botão (5:10–6:20)

Quem fala: integrante 3 (ou integrante 1, se grupo de dois).

“Em seguida podemos atualizar para *pago*, *separado*, *enviado* e *entregue*. Cada atualização gera um novo evento e uma mensagem. Existe uma regra importante: não se pode pular diretamente de recebido para entregue; a API responde com erro 409. No Telegram, o cliente recebe um botão *Confirmar ciência*, que volta pelo webhook. Esse clique é registrado como *respondida* — diferente de leitura, que o Telegram não informa.”

Ações na tela: *avançar quatro etapas, mostrar no Telegram, clicar no botão e atualizar a tabela de mensagens. Como teste opcional, tentar atualizar para uma etapa incorreta e explicar o HTTP 409.*

Slide 7 — Segurança, banco e testes (6:20–7:35)

Quem fala: integrante 3.

“Usamos o cabeçalho secreto do Telegram para validar a origem do webhook. Requisições sem o segredo correto recebem HTTP 401. Também armazenamos o `update_id` para evitar duplicidade quando a plataforma reenviar um evento. O banco contém clientes, produtos, pedidos, itens, histórico, vínculos e mensagens. O envio passa pelos estados *enfileirada*, *enviada* e *falha*, e a confirmação registra *respondida*. Além disso, há testes automatizados de regras, CEP e integração simulada com a Bot API.”

Ações na tela: *mostrar arquivo de migrations, terminal com `npm test` e, se possível, resposta 401/200 em um cliente HTTP. O token deve ficar oculto.*

Slide 8 — Nuvem e encerramento (7:35–9:00)

Quem fala: integrante 1.

“Na publicação final, a arquitetura separa a aplicação e o banco: a API fica na EC2 e o PostgreSQL no RDS, acessível apenas à aplicação por regras de rede. O Route 53 aponta nosso domínio para a máquina, e o Caddy faz HTTPS com certificado público. Também documentamos monitoramento e cuidado com o crédito de 200 dólares. Nosso resultado é uma base de e-commerce com rastreamento automatizado de pedidos, que demonstra API própria, API pública externa, bot, webhooks, segurança e persistência.”

Se a AWS ainda não foi implantada: substitua a primeira frase por “**Planejamos a publicação final na AWS** ...” e apresente o diagrama e passos como **plano**, não como resultado executado.

“Obrigado pela atenção. Agora podemos responder às perguntas.”

Perguntas prováveis do professor e respostas

Por que Telegram e não WhatsApp? O enunciado determina Telegram/Discord; Telegram simplifica o opt-in 1 para 1 e o webhook HTTPS.

Qual o papel da segunda API? O ViaCEP valida e completa campos do endereço, além de ser consultado novamente ao criar o pedido.

Como funciona a autenticação do webhook? O `setWebhook` configura um segredo, recebido em `X-Telegram-Bot-Api-Secret-Token`; a aplicação compara o valor e rejeita cabeçalhos incorretos com 401.

Como o sistema evita notificações duplicadas? O update do Telegram tem chave primária `update_id` e é salvo em `webhook_updates`; eventos processados não geram novo processamento. Para saída, há fila com tentativas e registro do `message_id`. Em falhas de rede após o Telegram aceitar a mensagem, ainda pode ocorrer duplicação na borda: limitação de sistemas distribuídos, que deve ser monitorada.

Por que não mostrar “lido”? Telegram/Discord não disponibilizam recibo de leitura via Bot API. A prova de ação é o clique no botão, marcado como `respondida`.

Como o valor do pedido é protegido? O servidor obtém os preços do catálogo no PostgreSQL e grava o valor do momento da compra em `itens_pedido`.

Qual a função das migrations? Versionar o esquema do banco, criando tabelas/índices com controle do que já foi aplicado, tanto local quanto no RDS.

Como impede outros usuários de consultar pedidos alheios pelo bot? `/pedido NUMERO` é consultado com filtro pelo `usuario_id` vinculado ao `chat_id` de origem.

O projeto já está implantado? Responda conforme evidência real: código preparado não equivale a serviço publicado. Se ainda não estiver, explique explicitamente o que falta.

O sistema processa pagamentos? Não. “Pago” é um estágio atualizado pelo operador, não uma cobrança bancária integrada. É um MVP focado em acompanhamento de status.

Checklist pré-apresentação

- ☐ Computador com Docker Desktop em execução; `/health` retornando `ok`.
- ☐ Banco migrado; `npm test` rodando na máquina do grupo.
- ☐ Usuário Telegram real vinculado, internet funcional, webhook configurado.
- ☐ Um cliente/produto/pedido de teste criados previamente para contingência.
- ☐ Chave administrativa e token fora das capturas e dos slides.
- ☐ Se houver implantação, domínio HTTPS e screenshots AWS reais preparados.
- ☐ Ensaiar a divisão de falas e as telas; ter prints de backup do fluxo **realmente testado**.